

```
# perl regex1.pl file1.txt
I found a match at: UNIX IS AN OPERATING SYSTEM
# perl regex2.pl file1.txt
UNIX IS AN OPERATING SYSTEM
# perl regex3.pl file1.txt
unix is an operating system
Unix is an Operating System
UNIX IS AN OPERATING SYSTEM
# perl regex4.pl file1.txt
unix
Unix
UNIX
```

Figure 3: Output of Perl scripts with regular expressions

```
if(/(UNIX)/i) {
    print "$1\n";
}
```

Changes are made in the fourth and fifth lines of the script *regex3.pl*. In the fourth line of code *if(/UNIX/i)* has been made *if(/(UNIX)/i)*. Here, parentheses are used for capturing matched strings in Perl. In the fifth line of code, we are printing *\$1*. Now, what is *\$1*? The special variable *\$1* holds the instance of a regular expression match. Figure 3 shows the output while processing *file1.txt* using the Perl scripts *regex1.pl*, *regex2.pl*, *regex3.pl* and *regex4.pl*.

Metacharacters in Perl

The following characters: . (dot), ^ (caret), \$ (dollar), * (asterisk), + (plus), ? (question mark), { (opening curly bracket), } (closing curly bracket), [(opening square bracket),] (closing square bracket), \ (backslash), | (pipe), ((opening parenthesis) and) (closing parenthesis) are special symbols used in Perl regular expressions. These are the same set of metacharacters used in Python regular expressions. The meanings of these metacharacters are the same in both Python and Perl, and a somewhat detailed discussion can be found in the previous article in this series. So if the meanings of the metacharacters are the same, what is the difference between the regular expressions *(abc){2,}* and *abc{2,}*? The regular expression syntax *{2,}* mandates at least two occurrences of the preceding regular expression. If so, why are the two different? The regular expression *(abc){2,}* matches the strings in the set *{abccabc, abccabcc, abccabcc, ...}* whereas *abc{2,}* matches the strings in the set *{abcc, abccc, abccccc, ...}*. This is due to the effect of the grouping operator opening and closing parentheses in the regular expression *(abc){2,}*. To test the effect of these regular expressions, let us use a text file called *file2.txt* with the following content.

```
abc
abcc
abccc
abccccc
abc
abccabc
abccabcc
abccabccabcc
```

```
# perl -ne 'if(/abc{2,}/){print "$_" }' file2.txt
abcc
abccc
abccccc
# perl -ne 'if(/(abc){2,}/){print "$_" }' file2.txt
abccabc
abccabcc
abccabccabcc
```

Figure 4: Use of metacharacters in Perl

Try the commands:

```
perl -ne 'if(/abc{2,}/){print "$_" }' file2.txt
```

and...

```
perl -ne 'if(/(abc){2,}/){print "$_" }' file2.txt.
```

The output of these two commands is shown in Figure 4.

Differences between Perl and Python metacharacters

Though the metacharacters in Perl and Python are almost the same, there are far more metasympols in Perl. An example of a metasympol is *\d*, which denotes a decimal digit. Metasympols like *\v*, *\W*, *\h*, *\H*, etc, are used to control the behaviour of whitespace characters. It is not possible to describe all the metasympols in this article, but keep in mind that many of them won't ever be used in the programming lifetime of even expert programmers. Perl regular expressions are not only powerful but the interaction between Perl variables and regular expressions is very easy. For example, a regular expression can be stored inside a variable like *\$s1 = "\a.*"*. To test this feature of Perl regular expressions, let us consider the text file *file3.txt* with the following content.

```
aaaa
bbbb
cccc
aabb
aacc
bbaa
bbcc
ccaa
ccbb
```

Now, execute the command:

```
perl -ne '$s1="\a.*"; if(/$s1/) { print $_ }' file3.txt.
```

The strings *aaaa*, *aabb* and *aacc* are selected because the regular expression matches the lines of text that start with an *a*. Similarly, try the command:

```
perl -ne '$s2=".*\c$"; if(/$s2/) { print $_ }' file3.txt.
```

This will select the strings *cccc*, *aacc* and *bbcc* because